

Homework 7: Computed Torque Control of a 4-DOF Manipulator

Step-by-Step Solution Guide

Nico_Robo_control

June 2026

Contents

1	Problem Overview	3
1.1	Assignment Tasks	3
2	Kinematic Modeling	3
2.1	Joint Twists (Screw Axes)	3
2.2	Forward Kinematics via Product of Exponentials	4
2.3	The 6×6 Adjoint Transformation	5
3	Dynamic Modeling	5
3.1	Link Inertial Parameters	5
3.2	Spatial Inertia Matrices	5
3.3	Equations of Motion	6
3.4	Computation of $\mathbf{M}(\boldsymbol{\theta})$ via Screw Theory	6
3.5	Computation of $\mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$	6
3.6	Computation of $\mathbf{N}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$	7
3.7	Code Implementation	7
3.8	The Coupling Matrix l_M	8
4	Part (a): Computed Torque Controller	8
4.1	Control Law	8
4.2	Choosing Gains	9
4.3	The Rounded Step Trajectory	9
4.4	Simulink Implementation	11
4.5	Verification	12
5	Part (b): Robustness Study	12
5.1	Methodology	12
5.2	Implementation in Simulink	12
5.3	What to Observe and Report	13

6	Part (c): PD Control Scheme	13
6.1	PD Control Law	13
6.2	Simulink Implementation	14
6.3	Gain Tuning for PD	14
6.4	What to Compare and Report	14
7	Part (d): Torque Saturation	15
7.1	Problem Formulation	15
7.2	Simulink Implementation	15
7.3	Effects of Saturation	16
7.4	Anti-Windup Strategies (Optional Investigation)	16
7.5	What to Report	16
8	Summary of the Complete Simulink Model	17
9	Expected Plots and Deliverables	17
10	Key Formulas Quick Reference	17
11	Troubleshooting Tips	18
12	Conclusion	18

1 Problem Overview

We consider a **four-degree-of-freedom** robotic manipulator with three revolute joints (joints 1–3) and one prismatic joint (joint 4), exactly as described in **MLS Example 4.4** (p. 177). The manipulator is shown schematically in Figure 1 in its reference (zero) configuration.

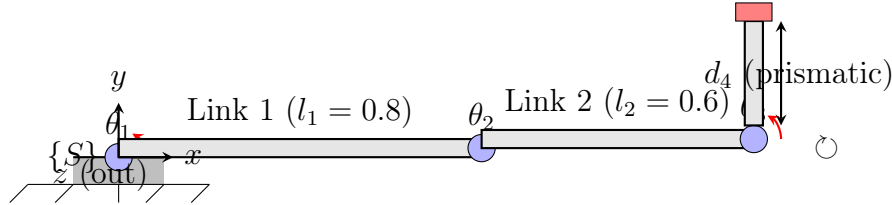


Figure 1: Schematic of the 4-DOF manipulator (3R + 1P) in its reference configuration. All three revolute joint axes are parallel to the z -axis; the prismatic joint translates along z .

1.1 Assignment Tasks

- (a) Build a **SIMULINK** model of a **computed torque controller** tracking a desired joint trajectory $\theta_d(t)$ in the form of a **rounded step function**. Desired final values:

$$\theta_{d1} \rightarrow 2 \text{ rad}, \quad \theta_{d2} \rightarrow 1 \text{ rad}, \quad \theta_{d3} \rightarrow 0.5 \text{ rad}, \quad d_{d4} \rightarrow 0.08 \text{ m}.$$

- (b) Study the **tracking robustness** of computed torque control by perturbing the plant's inertia parameters by 5–10% while keeping the controller at nominal values.
- (c) Implement and evaluate a **PD control** scheme for comparison.
- (d) Investigate **torque saturation** with limits $|\tau| \leq [6000, 2000, 1000, 100]^\top \text{ Nm}$ (or N).

2 Kinematic Modeling

2.1 Joint Twists (Screw Axes)

Using the **product of exponentials (PoE)** formulation, each joint is represented by a twist (screw axis) $\xi_i \in \mathbb{R}^6$ expressed in the spatial frame $\{S\}$.

For a **revolute** joint with axis ω and a point $\mathbf{q}$ on the axis:

$$\xi_i = \begin{bmatrix} -\hat{\omega}_i \mathbf{q}_i \\ \omega_i \end{bmatrix} \quad \text{where} \quad \text{hat}(\mathbf{v}) = \begin{bmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{bmatrix}.$$

For a **prismatic** joint with translation direction $\mathbf{v}$:

$$\xi_i = \begin{bmatrix} \mathbf{v}_i \\ \mathbf{0}_{3 \times 1} \end{bmatrix}.$$

Joint Twists ξ_i

$$\text{Joint 1 (R, about } z \text{ at origin): } \omega_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \mathbf{q}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \xi_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\text{Joint 2 (R, about } z \text{ at } [0, l_1, 0]^\top): \omega_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \mathbf{q}_2 = \begin{bmatrix} 0 \\ l_1 \\ 0 \end{bmatrix} \Rightarrow \xi_2 = \begin{bmatrix} l_1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\text{Joint 3 (R, about } z \text{ at } [0, l_1 + l_2, 0]^\top): \omega_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \mathbf{q}_3 = \begin{bmatrix} 0 \\ l_1 + l_2 \\ 0 \end{bmatrix} \Rightarrow \xi_3 = \begin{bmatrix} l_1 + l_2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\text{Joint 4 (P, along } z): \mathbf{v}_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \Rightarrow \xi_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

With numerical values $l_1 = 0.8 \text{ m}$, $l_2 = 0.6 \text{ m}$:

$$\xi_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad \xi_2 = \begin{bmatrix} 0.8 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad \xi_3 = \begin{bmatrix} 1.4 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad \xi_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

2.2 Forward Kinematics via Product of Exponentials

The homogeneous transformation from the spatial frame to link i is:

$$\mathbf{g}_{sl_i}(\boldsymbol{\theta}) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} \dots e^{\hat{\xi}_i \theta_i} \mathbf{g}_{sl_i}(0)$$

where $\mathbf{g}_{sl_i}(0)$ is the reference configuration of link i , and $\hat{\xi}$ is the 4×4 matrix representation of twist ξ in $\mathfrak{se}(3)$:

$$\hat{\xi} = \begin{bmatrix} \hat{\omega} & \mathbf{v} \\ \mathbf{0} & 0 \end{bmatrix}.$$

Reference Configurations $\mathbf{g}_{sl_i}(0)$

The reference (zero-angle) link frames are given by pure translations (no rotation):

$$\mathbf{g}_{sl_1}(0) = \begin{bmatrix} \mathbf{I} & [0, r_1, 0.3]^\top \\ \mathbf{0} & 1 \end{bmatrix}, \quad r_1 = l_1/2 = 0.4$$

$$\mathbf{g}_{sl_2}(0) = \begin{bmatrix} \mathbf{I} & [0, l_1 + r_2, 0.4]^\top \\ \mathbf{0} & 1 \end{bmatrix}, \quad r_2 = l_2/2 = 0.3$$

$$\mathbf{g}_{sl_3}(0) = \begin{bmatrix} \mathbf{I} & [0, l_1 + l_2, 0.5]^\top \\ \mathbf{0} & 1 \end{bmatrix}$$

$$\mathbf{g}_{sl_4}(0) = \begin{bmatrix} \mathbf{I} & [0, l_1 + l_2, l_0]^\top \\ \mathbf{0} & 1 \end{bmatrix}, \quad l_0 = 0.2$$

2.3 The 6×6 Adjoint Transformation

The adjoint map $\text{Ad}_{\mathbf{g}} \in \mathbb{R}^{6 \times 6}$ transforms twists between frames. For $\mathbf{g} = [\mathbf{R}, \mathbf{p}; \mathbf{0}, 1]$:

$$\text{Ad}_{\mathbf{g}} = \begin{bmatrix} \mathbf{R} & \hat{\mathbf{p}}\mathbf{R} \\ \mathbf{0} & \mathbf{R} \end{bmatrix}$$

This is implemented in the code as `adjmg.m` and `MCNsim.m` (lines 160–172).

3 Dynamic Modeling

3.1 Link Inertial Parameters

Table 1: Link Masses and Inertias

Link	Mass (kg)	I_{xx} (kg·m ²)	I_{yy} (kg·m ²)	I_{zz} (kg·m ²)
1	$m_1 = 50$	$m_1 l_1^2 / 12 = 2.667$	$0.05 I_{x1} = 0.133$	$I_{x1} = 2.667$
2	$m_2 = 30$	$m_2 l_2^2 / 12 = 0.900$	$0.05 I_{x2} = 0.045$	$I_{x2} = 0.900$
3	$m_3 = 20$	$2 I_{z3} = 0.180$	$2 I_{z3} = 0.180$	$0.1 I_{z2} = 0.090$
4	$m_4 = 5$	$2 I_{z4} = 0.108$	$2 I_{z4} = 0.108$	$0.6 I_{z3} = 0.054$

The **generalized inertia matrix** of link i in its own body frame is the 6×6 matrix:

$$\hat{\mathbf{M}}_i = \begin{bmatrix} m_i \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_i \end{bmatrix}, \quad \mathbf{I}_i = \text{diag}(I_{xi}, I_{yi}, I_{zi}).$$

3.2 Spatial Inertia Matrices

Each body inertia is transformed to the spatial frame $\{S\}$ using the adjoint transformation:

$$\hat{\mathbf{M}}_i^s = \text{Ad}_{\mathbf{g}_{sl_i}(0)}^{-\top} \hat{\mathbf{M}}_i \text{Ad}_{\mathbf{g}_{sl_i}(0)}^{-1}.$$

This is stored in the 3D array `calMp(:, :, i)`.

3.3 Equations of Motion

The manipulator dynamics in standard form:

$$\mathbf{M}(\boldsymbol{\theta}) \ddot{\boldsymbol{\theta}} + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) \dot{\boldsymbol{\theta}} + \mathbf{N}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) = \boldsymbol{\tau}$$

where:

- $\mathbf{M}(\boldsymbol{\theta}) \in \mathbb{R}^{4 \times 4}$ — **mass/inertia matrix** (symmetric, positive definite)
- $\mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) \in \mathbb{R}^{4 \times 4}$ — **Coriolis/centrifugal matrix**
- $\mathbf{N}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) \in \mathbb{R}^4$ — **gravity + friction vector**
- $\boldsymbol{\tau} \in \mathbb{R}^4$ — **joint torques/forces**

3.4 Computation of $\mathbf{M}(\boldsymbol{\theta})$ via Screw Theory

Using the **Christoffel symbol** approach and the spatial inertia formulation:

$$M_{ij}(\boldsymbol{\theta}) = \sum_{l=\max(i,j)}^n \boldsymbol{\xi}_i^\top \mathbf{A}_{li}^\top \hat{\mathbf{M}}_l^s \mathbf{A}_{lj} \boldsymbol{\xi}_j$$

where $\mathbf{A}_{ij} \in \mathbb{R}^{6 \times 6}$ are the adjoint maps between joints:

$$\mathbf{A}_{ij} = \begin{cases} \text{Ad}_{\mathbf{g}_{ji}}^{-1}, & i > j \\ \mathbf{I}_6, & i = j \\ \mathbf{0}, & i < j \end{cases}, \quad \mathbf{g}_{ji} = \mathbf{g}_j^{-1} \mathbf{g}_i.$$

3.5 Computation of $\mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$

The Coriolis matrix is obtained from the partial derivatives of $\mathbf{M}$ via **Christoffel symbols of the first kind**:

$$C_{ij} = \sum_{k=1}^n \Gamma_{ijk} \dot{\theta}_k, \quad \Gamma_{ijk} = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial \theta_k} + \frac{\partial M_{ik}}{\partial \theta_j} - \frac{\partial M_{kj}}{\partial \theta_i} \right).$$

The partial derivative $\partial M_{ij} / \partial \theta_k$ is computed analytically:

$$\begin{aligned} \frac{\partial M_{ij}}{\partial \theta_k} = \sum_{l=\max(i,j)}^n & \left[(\text{ad}_{\mathbf{A}_{ki} \boldsymbol{\xi}_i} \boldsymbol{\xi}_k)^\top \mathbf{A}_{lk}^\top \hat{\mathbf{M}}_l^s \mathbf{A}_{lj} \boldsymbol{\xi}_j \right. \\ & \left. + \boldsymbol{\xi}_i^\top \mathbf{A}_{li}^\top \hat{\mathbf{M}}_l^s \mathbf{A}_{lk} (\text{ad}_{\mathbf{A}_{kj} \boldsymbol{\xi}_j} \boldsymbol{\xi}_k) \right], \end{aligned}$$

valid for $k \leq l$, and zero otherwise. Here $\text{ad}_{\boldsymbol{\xi}}$ is the 6×6 Lie bracket (adjoint) matrix:

$$\text{ad}_{\boldsymbol{\xi}} = \begin{bmatrix} \hat{\boldsymbol{\omega}} & \hat{\mathbf{v}} \\ \mathbf{0} & \hat{\boldsymbol{\omega}} \end{bmatrix}.$$

3.6 Computation of $\mathbf{N}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$

The vector $\mathbf{N}$ consists of two parts:

1. Gravity Term

The gradient of the gravitational potential energy:

$$\frac{\partial V}{\partial \theta_i} = - \sum_{l=i}^n \boldsymbol{\xi}_l^\top \text{Ad}_{\mathbf{g}_{0l}}^{-\top} \hat{\mathbf{M}}_l^s \mathbf{G}_0$$

where $\mathbf{G}_0 \in \mathbb{R}^6$ is the **gravity twist** in the spatial frame:

$$\mathbf{G}_0 = \begin{bmatrix} -\mathbf{g}_{\text{accel}} \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 9.81 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

(The sign follows the code convention: $-\mathbf{g}_{\text{accel}}$ in the linear part.)

2. Viscous Friction Term

$$N_{\text{fric},i} = \beta_i \dot{\theta}_i$$

with $\boldsymbol{\beta} = [1.0, 0.5, 0.4, 1.0]$ (Nm·s/rad for revolute, N·s/m for prismatic).

Total:

$$\mathbf{N}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) = -\nabla_{\boldsymbol{\theta}} V(\boldsymbol{\theta}) + \text{diag}(\boldsymbol{\beta}) \dot{\boldsymbol{\theta}}.$$

3.7 Code Implementation

The dynamics computation is implemented in `MCN.m` (and its self-contained variant `MCNsim.m` for Simulink). The `MCNsim.m` file is used inside Simulink MATLAB Function blocks because it contains `adjmg` as a local nested function (no external dependencies).

Listing 1: `MCNsim.m` — Dynamics computation (self-contained for Simulink)

```

1 function [Ms, Cs, Ns] = MCNsim(xi, calMp, lM, beta, m4g, theta,
2   dtheta)
3     n = length(theta);
4     % Step 1: FK --- compute all g_{0i}
5     g = cell(1, n+1); g{1} = eye(4);
6     for i = 1:n
7         g{i+1} = g{i} * expm(hat_se3(xi(:,i)) * theta(i));
8     end
9     % Step 2: Adj maps A_{ij}
10    A = cell(n,n);
11    for i = 1:n, for j = 1:n
12        if i > j, A{i,j} = inv(adjmg(inv(g{j+1}) * g{i+1}));
13        elseif i == j, A{i,j} = eye(6);

```

```

13         else,                A{i,j} = zeros(6);
14         end
15     end, end
16     % Step 3: M matrix
17     for i = 1:n, for j = 1:n
18         M_ij = 0;
19         for l = lM(i,j):n
20             M_ij = M_ij + xi(:,i)' * A{l,i}' * calMp(:, :, l) * A{l,j}
                * xi(:,j);
21         end
22         Ms(i,j) = M_ij;
23     end, end
24     % Step 4: C matrix via Christoffel symbols
25     for i = 1:n, for j = 1:n
26         C_ij = 0;
27         for k = 1:n
28             C_ij = C_ij + 0.5 * (dM(i,j,k) + dM(i,k,j) - dM(k,j,i))
                * dtheta(k);
29         end
30         Cs(i,j) = C_ij;
31     end, end
32     % Step 5: N vector (gravity + friction)
33     grad_V = compute_gravity_potential_gradient(xi, calMp, g, n);
34     Ns = -grad_V + beta(:) .* dtheta(:);
35 end

```

3.8 The Coupling Matrix l_M

The lM matrix encodes which links contribute to each (i, j) pair of the mass matrix:

$$l_M(i, j) = \max(i, j).$$

For $n = 4$:

$$l_M = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 2 & 3 & 4 \\ 3 & 3 & 3 & 4 \\ 4 & 4 & 4 & 4 \end{bmatrix}.$$

4 Part (a): Computed Torque Controller

4.1 Control Law

The **computed torque** (feedback linearization) controller applies:

$$\tau = M(\theta) \ddot{\theta}^* + C(\theta, \dot{\theta}) \dot{\theta} + N(\theta, \dot{\theta})$$

where $\ddot{\theta}^*$ is a virtual input chosen to achieve tracking. Substituting into the dynamics yields the **double-integrator** error dynamics:

$$\ddot{\theta} = \ddot{\theta}^*.$$

A **PD+feedforward** law for $\ddot{\theta}^*$ is:

$$\ddot{\theta}^* = \ddot{\theta}_d + \mathbf{K}_v(\dot{\theta}_d - \dot{\theta}) + \mathbf{K}_p(\theta_d - \theta)$$

where $\mathbf{K}_p = \text{diag}(k_{p1}, \dots, k_{p4})$ and $\mathbf{K}_v = \text{diag}(k_{v1}, \dots, k_{v4})$ are positive definite gain matrices.

This yields the **linear error dynamics**:

$$\ddot{\mathbf{e}} + \mathbf{K}_v \dot{\mathbf{e}} + \mathbf{K}_p \mathbf{e} = \mathbf{0}, \quad \mathbf{e}(t) = \theta_d(t) - \theta(t).$$

4.2 Choosing Gains

For a desired closed-loop natural frequency ω_n and damping ratio ζ :

$$k_{pi} = \omega_{n,i}^2, \quad k_{vi} = 2\zeta_i \omega_{n,i}.$$

Suggested initial values:

$$\mathbf{K}_p = \text{diag}(2500, 2500, 2500, 1600), \quad \mathbf{K}_v = \text{diag}(100, 100, 100, 80)$$

(corresponding to $\omega_n \approx 50 \text{ rad/s}$ for revolute, 40 for prismatic, with $\zeta = 1$).

4.3 The Rounded Step Trajectory

Since computed torque control requires θ_d to be **twice differentiable**, a **quintic polynomial** rounded step is used:

$$r(t) = \begin{cases} 0, & t \leq t_0 \\ 10 \left(\frac{t-t_0}{\Delta t} \right)^3 - 15 \left(\frac{t-t_0}{\Delta t} \right)^4 + 6 \left(\frac{t-t_0}{\Delta t} \right)^5, & t_0 < t < t_0 + \Delta t \\ 1, & t \geq t_0 + \Delta t \end{cases}$$

Properties of the Quintic Rounded Step

- $r(t_0) = 0, r(t_0 + \Delta t) = 1$
- $\dot{r}(t_0) = \dot{r}(t_0 + \Delta t) = 0$
- $\ddot{r}(t_0) = \ddot{r}(t_0 + \Delta t) = 0$
- The function is $\mathcal{C}^2$ continuous everywhere.

The transition parameters (from `initialize_simulink.m`):

$$t_0 = 1.0 \text{ s}, \quad \Delta t = 0.2 \text{ s},$$

so the step starts at $t = 1.0 \text{ s}$, ramps from $t = 1.0$ to $t = 1.2 \text{ s}$, then holds constant.

The desired joint trajectories are obtained by scaling:

$$\theta_d(t) = \begin{bmatrix} 2 \\ 1 \\ 0.5 \\ 0.08 \end{bmatrix} \cdot r(t), \quad \dot{\theta}_d(t) = \begin{bmatrix} 2 \\ 1 \\ 0.5 \\ 0.08 \end{bmatrix} \cdot \dot{r}(t), \quad \ddot{\theta}_d(t) = \begin{bmatrix} 2 \\ 1 \\ 0.5 \\ 0.08 \end{bmatrix} \cdot \ddot{r}(t).$$

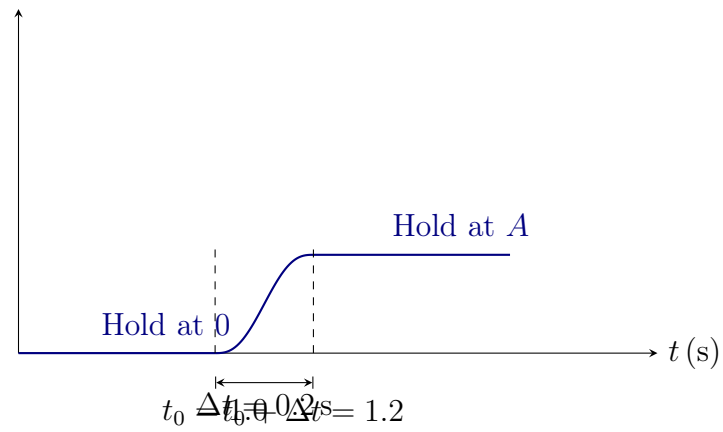


Figure 2: Quintic rounded step function $A \cdot r(t)$ with amplitude A .

4.4 Simulink Implementation

Step-by-Step Simulink Construction

S1. Copy the rounded step block from `MLS4_4_rounded_step_inputR2024a.slx` into your new model. This block outputs $[y, \dot{y}, \ddot{y}]^\top$ where y is the quintic rounded step.

S2. Scale the outputs: Multiply the $y, \dot{y}, \ddot{y}$ signals by the amplitude vector $[2, 1, 0.5, 0.08]$ to obtain $\theta_d, \dot{\theta}_d, \ddot{\theta}_d$.

S3. Implement two MATLAB Function blocks for dynamics:

- **Controller dynamics block:** Contains `MCNsim` with **nominal** parameters (call it `MCN_ctrl`). Inputs: $\theta, \dot{\theta}$. Outputs: $\mathbf{M}_c, \mathbf{C}_c, \mathbf{N}_c$.
- **Plant dynamics block:** Contains `MCNsim` with (potentially perturbed) parameters (call it `MCN_plant`). Inputs: $\theta, \dot{\theta}$. Outputs: $\mathbf{M}_p, \mathbf{C}_p, \mathbf{N}_p$.

(Initially, both use identical nominal parameters.)

S4. Compute $\ddot{\theta}^*$:

$$\ddot{\theta}^* = \ddot{\theta}_d + \mathbf{K}_v(\dot{\theta}_d - \dot{\theta}) + \mathbf{K}_p(\theta_d - \theta)$$

Use matrix gain blocks or a MATLAB Function block.

S5. Compute control torque τ :

$$\tau = \mathbf{M}_c \ddot{\theta}^* + \mathbf{C}_c \dot{\theta} + \mathbf{N}_c$$

S6. Solve forward dynamics using the plant model:

$$\ddot{\theta} = \mathbf{M}_p^{-1}(\tau - \mathbf{C}_p \dot{\theta} - \mathbf{N}_p)$$

Use a MATLAB Function block that inverts $\mathbf{M}_p$ and computes the acceleration.

S7. Integrate twice to get $\dot{\theta}$ and θ :

- First integrator: $\ddot{\theta} \rightarrow \dot{\theta}$
- Second integrator: $\dot{\theta} \rightarrow \theta$

S8. Initial conditions: Set $\theta(0) = [\pi/4, \pi/6, \pi/2.5, -0.04]^\top$ and $\dot{\theta}(0) = \mathbf{0}$.

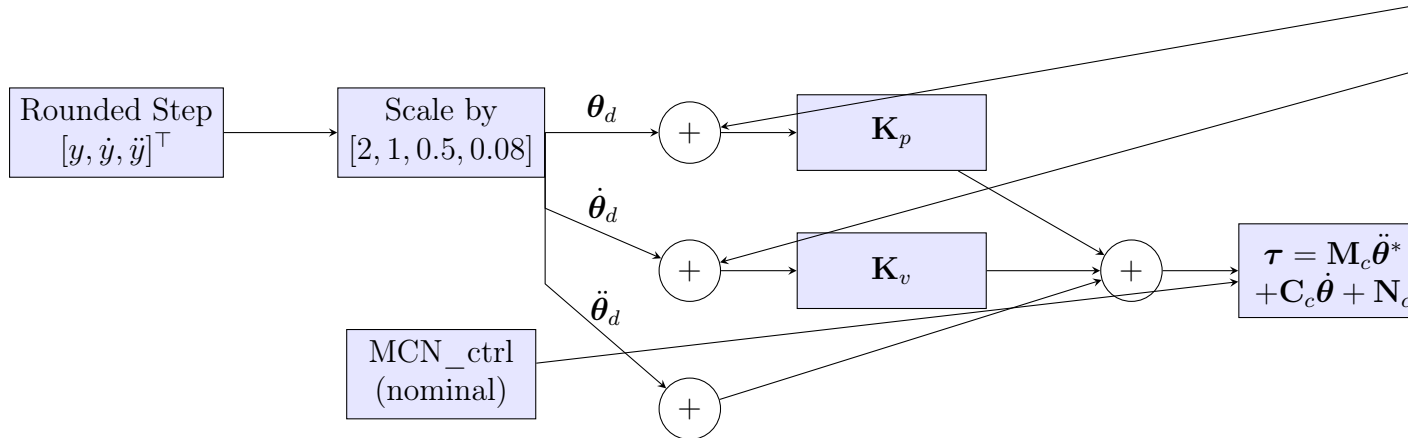


Figure 3: High-level Simulink block diagram for the computed torque controller. Two MCN blocks are used: one for the controller (nominal) and one for the plant (possibly perturbed).

4.5 Verification

Verification Steps

1. Run the simulation for 3–5 seconds.
2. Check that all joints converge to their desired final values: $\theta_1 \rightarrow 2$, $\theta_2 \rightarrow 1$, $\theta_3 \rightarrow 0.5$, $d_4 \rightarrow 0.08$.
3. Verify that tracking errors $\mathbf{e}(t) = \boldsymbol{\theta}_d(t) - \boldsymbol{\theta}(t)$ converge to zero.
4. Check that the torque $\boldsymbol{\tau}$ is physically reasonable.

5 Part (b): Robustness Study

5.1 Methodology

To assess robustness, we introduce a **mismatch** between the **plant** (actual physical robot) and the **controller** (which assumes nominal parameters).

Perturbation Strategy

Plant only: Multiply all entries of each link's spatial inertia matrix $\hat{\mathbf{M}}_i^s$ by $(1 + \delta)$ in the MCN_plant block, where $\delta \in \{-0.10, -0.05, +0.05, +0.10\}$.

Controller: Keep the MCN_ctrl block using the original (nominal) $\hat{\mathbf{M}}_i^s$.

5.2 Implementation in Simulink

In the MCNsim function used by the plant, modify the calMp array as:

```
1 % In MCN_plant only:
```

```

2 | delta = 0.05; % or -0.05, 0.10, -0.10
3 | calMp_perturbed = calMp * (1 + delta);
4 | % Use calMp_perturbed in the dynamics computation

```

Alternatively, pass a `delta` parameter as an additional input to your MATLAB Function block.

5.3 What to Observe and Report

1. **Tracking error:** Plot $\mathbf{e}(t)$ for $\delta = 0$ (baseline) vs. $\delta = \pm 5\%$, $\pm 10\%$. The error should **not converge to zero** when there is parameter mismatch — this is the key weakness of computed torque control (it relies on exact model cancellation).
2. **Steady-state error:** Even at steady state, a non-zero error persists due to incorrect gravity compensation.
3. **Transient behavior:** Overshoot and settling time change with parameter perturbation.
4. **Effect direction:** Positive δ (heavier robot than modeled) vs. negative δ (lighter robot) produce different error patterns.

Expected Results

- With **no perturbation** ($\delta = 0$): perfect asymptotic tracking, $\mathbf{e}(t) \rightarrow \mathbf{0}$.
- With **perturbation** ($\delta \neq 0$): non-zero steady-state error; the larger $|\delta|$, the larger the error.
- The prismatic joint (joint 4) is typically most sensitive due to direct gravity loading.

6 Part (c): PD Control Scheme

6.1 PD Control Law

The **PD+gravity compensation** controller is:

$$\boldsymbol{\tau}_{\text{PD}} = \mathbf{K}_p(\boldsymbol{\theta}_d - \boldsymbol{\theta}) + \mathbf{K}_v(\dot{\boldsymbol{\theta}}_d - \dot{\boldsymbol{\theta}}) + \mathbf{N}(\boldsymbol{\theta})$$

Compared to computed torque, the PD controller:

- **Does not use** $\mathbf{M}(\boldsymbol{\theta})$ or $\mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$ in the feedback path (only $\mathbf{N}$ for gravity compensation).
- Is **simpler** to implement and computationally cheaper.
- Has **nonlinear** closed-loop error dynamics (cannot perfectly cancel nonlinearities).
- **Does not require** $\ddot{\boldsymbol{\theta}}_d$ (only $\boldsymbol{\theta}_d$ and $\dot{\boldsymbol{\theta}}_d$).

The closed-loop dynamics under PD control:

$$\mathbf{M}(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}} + \mathbf{N}(\boldsymbol{\theta}) = \mathbf{K}_p\mathbf{e} + \mathbf{K}_v\dot{\mathbf{e}} + \mathbf{N}(\boldsymbol{\theta})$$

which does **not** reduce to a simple linear system.

6.2 Simulink Implementation

PD Controller Block Diagram — Simpler than computed torque

1. Remove the $\mathbf{M}_c$, $\mathbf{C}_c$, and $\mathbf{N}_c$ blocks.
2. Remove the feedforward $\ddot{\boldsymbol{\theta}}_d$ term.
3. Compute:

$$\boldsymbol{\tau} = \mathbf{K}_p(\boldsymbol{\theta}_d - \boldsymbol{\theta}) + \mathbf{K}_v(\dot{\boldsymbol{\theta}}_d - \dot{\boldsymbol{\theta}}) + \mathbf{N}_p(\boldsymbol{\theta})$$

where $\mathbf{N}_p$ is the gravity vector only (friction optional). Note: for PD only (no gravity comp), omit the $\mathbf{N}_p$ term.

4. Feed $\boldsymbol{\tau}$ directly to the forward dynamics (plant model).

6.3 Gain Tuning for PD

PD gains typically need to be **larger** than computed torque gains to compensate for the lack of model-based decoupling:

$$\mathbf{K}_p^{\text{PD}} \approx (2-5) \times \mathbf{K}_p^{\text{CT}}, \quad \mathbf{K}_v^{\text{PD}} \approx (2-5) \times \mathbf{K}_v^{\text{CT}}.$$

6.4 What to Compare and Report

Table 2: Comparison: Computed Torque vs. PD Control

Criterion	Computed Torque	PD (+ Gravity Comp)
Model dependency	Full $\mathbf{M}$, $\mathbf{C}$, $\mathbf{N}$	$\mathbf{N}$ only (or none)
Computational cost	High (matrix inversion)	Low
Tracking accuracy	Excellent (if model exact)	Moderate (nonlinear coupling)
Robustness to param. errors	Poor (relies on cancellation)	Good (no cancellation needed)
Gain tuning	Systematic (pole placement)	Heuristic / trial-and-error
$\ddot{\boldsymbol{\theta}}_d$ required?	Yes	No

Expected Results

- **Computed torque** achieves faster, more accurate tracking when the model is exact.
- **PD** has larger tracking errors during transients due to uncompensated dynamic coupling.
- **PD** is more robust to parameter uncertainty (the error degrades gracefully rather than catastrophically).
- With gravity compensation, PD achieves zero steady-state error for step references (by the passivity argument).
- Without gravity compensation, PD has steady-state error.

7 Part (d): Torque Saturation

7.1 Problem Formulation

Real actuators have finite torque/force capabilities. The saturation limits are:

$$|\tau_i| \leq \tau_{i,\max}, \quad \boldsymbol{\tau}_{\max} = \begin{bmatrix} 6000 \\ 2000 \\ 1000 \\ 100 \end{bmatrix} \text{ Nm (joints 1–3) / N (joint 4).}$$

When the computed torque controller demands τ_i beyond $\tau_{i,\max}$, the actual applied torque is **clipped**:

$$\tau_i^{\text{applied}} = \text{sat}(\tau_i^{\text{commanded}}, -\tau_{i,\max}, \tau_{i,\max}) = \begin{cases} \tau_{i,\max}, & \tau_i^{\text{commanded}} > \tau_{i,\max} \\ \tau_i^{\text{commanded}}, & |\tau_i^{\text{commanded}}| \leq \tau_{i,\max} \\ -\tau_{i,\max}, & \tau_i^{\text{commanded}} < -\tau_{i,\max} \end{cases}$$

7.2 Simulink Implementation

Saturation Block Placement

Insert a **Saturation** block (from Simulink → Discontinuities library) between the computed torque output and the forward dynamics plant input, with limits set to $\pm[6000, 2000, 1000, 100]$.

7.3 Effects of Saturation

Key Phenomena

1. **Integrator windup:** When the torque saturates, the error keeps accumulating in the integrator (if used), causing large overshoots when the system comes out of saturation. This is called **windup**.
2. **Slower response:** The system cannot accelerate as fast as commanded, so tracking is degraded during the transient.
3. **Potential instability:** If the controller relies on the exact cancellation of nonlinearities and the applied torque differs significantly from the commanded torque, the error dynamics are no longer linear and may become unstable.
4. **Step-size dependence:** Larger desired steps require larger torques; the saturation limits may be exceeded more severely.

7.4 Anti-Windup Strategies (Optional Investigation)

1. **Back-calculation:** Feed the difference ($\tau_{\text{applied}} - \tau_{\text{commanded}}$) back to reduce the integral action.
2. **Conditional integration:** Freeze the integrator when saturation is active.
3. **Reference governor:** Scale down $\ddot{\theta}^*$ when the commanded torque exceeds limits.

7.5 What to Report

1. Plot commanded vs. applied torques for each joint.
2. Compare tracking performance with and without saturation.
3. Identify which joints saturate first (typically joint 1, which supports the entire arm).
4. Investigate whether the saturation limits are adequate for the desired trajectory.
5. If windup is observed, suggest and test an anti-windup modification.

Expected Results

- Joint 1 (the base joint) requires the largest torque and is most likely to saturate first.
- Under saturation, tracking errors increase and may not converge to zero.
- The response becomes slower and may exhibit overshoot due to windup.
- With anti-windup, the overshoot is reduced but the speed is still limited by actuator capability.

8 Summary of the Complete Simulink Model

Controller side (runs at each time step):

1. Read $\theta, \dot{\theta}$ (from integrators)
2. Read $\theta_d, \dot{\theta}_d, \ddot{\theta}_d$ (from rounded step generator)
3. Compute $e = \theta_d - \theta, \dot{e} = \dot{\theta}_d - \dot{\theta}$
4. Compute: $\ddot{\theta}^* = \ddot{\theta}_d + K_v \dot{e} + K_p e$
5. Call $\text{MCN_ctrl}(\theta, \dot{\theta}) \rightarrow M_c, C_c, N_c$
6. Compute: $\tau = M_c \ddot{\theta}^* + C_c \dot{\theta} + N_c$
7. Apply saturation: $\tau_a = \text{sat}(\tau)$

Plant side (forward dynamics):

1. Call $\text{MCN_plant}(\theta, \dot{\theta}) \rightarrow M_p, C_p, N_p$
2. Compute: $\ddot{\theta} = M_p^{-1}(\tau_a - C_p \dot{\theta} - N_p)$
3. Integrate: $\dot{\theta} = \int \ddot{\theta} dt, \theta = \int \dot{\theta} dt$

Figure 4: Complete computational flow of the control+simulation system.

9 Expected Plots and Deliverables

For your report, include the following figures:

1. **Tracking performance (nominal case):** $\theta(t)$ vs. $\theta_d(t)$ for all 4 joints, plus tracking errors $e(t)$.
2. **Control torques (nominal):** $\tau(t)$ for all 4 joints.
3. **Robustness:** Tracking errors for $\delta = -10\%, -5\%, 0, +5\%, +10\%$ (overlaid or subplots).
4. **PD comparison:** Tracking performance of PD vs. computed torque (overlaid).
5. **Torque saturation:** Commanded vs. applied torque, and tracking error with/without saturation.
6. (Optional) **Anti-windup:** Comparison of tracking error with and without anti-windup under saturation.

10 Key Formulas Quick Reference

Computed Torque Control

$$\tau = M(\theta) [\ddot{\theta}_d + K_v \dot{e} + K_p e] + C(\theta, \dot{\theta}) \dot{\theta} + N(\theta, \dot{\theta})$$

$$e = \theta_d - \theta, \quad \dot{e} = \dot{\theta}_d - \dot{\theta}$$

PD Control with Gravity Compensation

$$\boldsymbol{\tau} = \mathbf{K}_p \mathbf{e} + \mathbf{K}_v \dot{\mathbf{e}} + \mathbf{N}_{\text{grav}}(\boldsymbol{\theta})$$

Forward Dynamics (Simulation)

$$\ddot{\boldsymbol{\theta}} = \mathbf{M}^{-1}(\boldsymbol{\theta}) [\boldsymbol{\tau} - \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) \dot{\boldsymbol{\theta}} - \mathbf{N}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})]$$

Quintic Rounded Step

$$r(t) = 10 \left(\frac{t - t_0}{\Delta t} \right)^3 - 15 \left(\frac{t - t_0}{\Delta t} \right)^4 + 6 \left(\frac{t - t_0}{\Delta t} \right)^5$$

for $t_0 < t < t_0 + \Delta t$, with $t_0 = 1.0 \text{ s}$, $\Delta t = 0.2 \text{ s}$.

11 Troubleshooting Tips

Common Pitfalls

1. **Algebraic loop:** If $\mathbf{M}$, $\mathbf{C}$, or $\mathbf{N}$ depend on $\ddot{\boldsymbol{\theta}}$, Simulink will report an algebraic loop. Ensure your MCN function takes only $(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})$ as inputs (not $\ddot{\boldsymbol{\theta}}$).
2. **Dimension mismatch:** All signals are 4×1 vectors. Use **Selector** blocks or **Demux/Mux** blocks carefully.
3. **Singular M:** $\mathbf{M}(\boldsymbol{\theta})$ is always invertible for this manipulator. If you get a singular matrix warning, check your inertia parameters.
4. **MATLAB Function block dependencies:** Use `MCNsim.m` (self-contained) inside Simulink MATLAB Function blocks, not `MCN.m` (has external dependencies).
5. **Step size too large:** If the solver diverges, reduce the maximum step size in Configuration Parameters (try $1e-3$ or smaller).
6. **Initial conditions:** The initial $\boldsymbol{\theta}$ must be set in the integrator blocks. The initial $\dot{\boldsymbol{\theta}}$ is typically zero.

12 Conclusion

This homework demonstrates the full pipeline of robot control system design:

1. **Kinematic and dynamic modeling** using screw theory (product of exponentials) provides an elegant, coordinate-free formulation suitable for symbolic and numerical computation.
2. **Computed torque control** achieves perfect tracking when the model is exact, by exactly canceling the nonlinear dynamics and imposing linear error dynamics.

3. **Robustness** is the Achilles' heel of computed torque control — parameter mismatch leads to non-zero tracking errors, motivating adaptive or robust control extensions.
4. **PD control** is simpler and more robust but sacrifices tracking performance due to uncompensated dynamic coupling.
5. **Actuator saturation** imposes practical limits that must be accounted for, potentially requiring anti-windup strategies.

End of Solution Guide